kNN-DBSCAN: algorithm, distributed implementation and correctness

A technical companion to Chen, Ruys and Biros, "KNN-DBSCAN: a DBSCAN in high dimensions"

knndbscan project
github.com/Prajwal-Prathiksh/knndbscan

November 2026 · knndbscan 0.2

Abstract. kNN-DBSCAN replaces DBSCAN's range queries by a k-nearest-neighbor graph, which can be computed approximately and at scale in high dimensions. This report distills the method of Chen, Ruys and Biros [1] for readers who want to understand, use or modify an implementation without first reading the paper: the definitions and how they relate to DBSCAN, the reduction of clustering to a spanning forest, Borůvka's algorithm on a directed kNN graph, and the distributed (MPI + OpenMP) algorithm. It adds what the paper does not need to say but an implementation must get right: numerical conventions, the cycles that directed graphs create in Borůvka's pointer jumping, the invariants the code depends on, and how correctness is verified. Every claim about the implementation is checked by the project's test suite.

Contents

1 Introduction 1
1.1 Notation 2
2 DBSCAN 2
3 kNN-DBSCAN 2
3.1 Classifying points from the k-NNG 2
3.2 Reachability through the graph 3
4 Clusters as spanning forests 4
4.1 From components to spanning trees 4
4.2 Borůvka's algorithm 4
4.3 Cycles in a directed graph 4
4.4 Minimum edges are not needed 5
5 The distributed algorithm 6
5.1 Data layout 6
5.2 The inexact minimum spanning tree 6
5.3 Stages 6
6 Numerical conventions and invariants 7
7 Verifying an implementation 8
8 Performance 9
9 Using it 9
References 10
A The algorithm in pictures 11
A.1 From points to clusters 11
A.2 Borůvka, round by round 11
A.3 Across ranks 12
A.4 Where implementations go wrong 12

1 Introduction

DBSCAN [2] clusters points by density: a point with at least M points within distance ε is a *core point*, core points within ε of each other belong to the same cluster, and points near a cluster's core join it as *border points*; the rest is *noise*. It needs no number of clusters, finds clusters of any shape and is robust to outliers. Its cost lies in the ε-neighborhoods: range queries are cheap with spatial trees in low dimensions but degrade towards $O(n^2)$ comparisons as the dimension grows, and every new value of ε repeats them.

kNN-DBSCAN [1] keeps DBSCAN's parameters and its classification of points, but connects core points through a k-nearest-neighbor graph (k-NNG) instead of ε-neighborhoods. A k-NNG can be approximated well in high dimensions (randomized projection trees [3], as in GOFMM [4]), is computed once for all ε and all $M \leq k+1$, and stores $O(nk)$ edges. Clustering the graph then reduces to computing a spanning forest, which parallelizes with Borůvka's algorithm in shared and distributed memory.

The report follows the path from definitions to code: Section 2 and Section 3 define both methods and relate them; Section 4 reduces clustering to a spanning forest and develops Borůvka's algorithm on a directed graph; Section 5 distributes it over MPI ranks; Section 6 collects the numerical conventions and invariants an implementation must keep; Section 7 describes how correctness is verified, with the defects this found in the original implementation; Section 8 summarizes performance. Appendix A shows each stage, and each defect, on a small example in pictures tied to the lines of the pseudocode.

> **Note.** Numbers such as "Theorem 3.4" refer to the paper. Statements here are restated in a single notation and sometimes split or sharpened; the proofs are sketches that convey why a result holds. The code is organized by the same stages: `docs/ALGORITHM.md` maps each one to its file and function.

1.1 Notation

Symbol	Meaning
$\mathcal{I}$	the set of n points
$d(p,q)$	distance between points p and q (Euclidean in the Python package)
ε, M	radius and minimum number of points (`eps`, `min_samples`); M counts the point itself
$\mathcal{N}_\varepsilon(p)$	ε-neighborhood $\{q \in \mathcal{I} : d(p,q) \leq \varepsilon\}$, including p
$\mathcal{N}_M(p)$	the M nearest points to p, including p
k	neighbors stored per point in the k-NNG, excluding the point itself
$\mathcal{I}_{\text{core}}, \mathcal{I}_{\text{border}}, \mathcal{I}_{\text{noise}}$	the three classes of points
P	number of MPI ranks

Table 1: Notation.

2 DBSCAN

> **Definition 1** (classification of points, Def. 3.1). Given ε and M, a point p is a *core point* if $|\mathcal{N}_\varepsilon(p)| \geq M$. A point that is not core is a *border point* if some core point q has $p \in \mathcal{N}_\varepsilon(q)$, and *noise* otherwise.

> **Definition 2** (reachability and DBSCAN clusters, Defs. 3.2–3.5). q is *directly ε-reachable* from p if p is core and $q \in \mathcal{N}_\varepsilon(p)$; *$\varepsilon$-reachable* if a chain of core points $p = p_0, p_1, \ldots, p_m$ links them by direct reachability, ending in q. A *DBSCAN cluster* is a nonempty set C that is closed under ε-reachability (if $p \in C$ is core and q is reachable from p, then $q \in C$) and connected (any two members are reachable from a common core point).

On core points, ε-reachability is an equivalence relation, so the core points of the clusters are its equivalence classes: the connected components of the graph that joins core points at distance at most ε. Border points are not uniquely placed: a border point within ε of core points of two clusters belongs to either. This ambiguity is inherent to DBSCAN and carries over to kNN-DBSCAN.

3 kNN-DBSCAN

3.1 Classifying points from the k-NNG

The first observation is that the classification of Definition 1 needs only the M nearest neighbors of each point.

Lemma 1 (Lemma 3.1).
1. p is a core point if and only if $\max_{q \in \mathcal{N}_M(p)} d(p,q) \le \varepsilon$, that is, if its $(M-1)$-th nearest neighbor other than itself lies within ε.
2. p is a border point if and only if p is not core and some core point $q \in \mathcal{N}_M(p)$ has $d(p,q) \le \varepsilon$.

Proof. (1) $\mathcal{N}_\varepsilon(p)$ contains at least M points exactly when the M-th closest point (counting p) is within ε. (2) If p is not core, its M-th closest point is farther than ε, so every point within ε of p, in particular a core point that makes p a border point, is among its M nearest. □

Consequently a k-NNG with $k \ge M-1$ classifies every point exactly as DBSCAN does, and the classification is the same whatever larger k was stored. In the stored rows, which list each point's neighbors in increasing distance without the point itself, p is core when the distance in column $M-1$ is at most ε.

3.2 Reachability through the graph

DBSCAN links core points that are within ε of each other. kNN-DBSCAN links them only through the graph:

Definition 3 (M-reachability and kNN-DBSCAN clusters, Defs. 3.6–3.9). q is *directly M-reachable* from p if p is core and $q \in \mathcal{N}_M(p)$ or $p \in \mathcal{N}_M(q)$. M-reachability, M-connectedness and *kNN-DBSCAN clusters* are defined from it exactly as their ε counterparts.

The relation is symmetrized: an edge counts if either endpoint lists the other. Two facts make it well behaved.

Lemma 2 (Lemmas 3.2 and 3.3). On core points, M-reachability is an equivalence relation, and it implies ε-reachability.

Proof. Symmetry comes from the symmetrized definition; reflexivity and transitivity from concatenating chains. If p is core then every $q \in \mathcal{N}_M(p)$ satisfies $d(p,q) \le \varepsilon$ by Lemma 1, so each M-step is an ε-step. □

So the core points of kNN-DBSCAN clusters are the connected components of the graph $G_{M,\,\text{core}}$ whose vertices are the core points and whose edges join p to its first $M-1$ neighbors when both are core. No distance test is needed on these edges: they are within ε automatically.

Theorem 1 (relation to DBSCAN, Theorem 3.4). With the same ε and M:
1. every kNN-DBSCAN cluster's core points lie in a single DBSCAN cluster, so kNN-DBSCAN finds at least as many clusters as DBSCAN;
2. if the numbers of clusters are equal, the two clusterings of the core points coincide.

Core, border and noise points are the same in both (Lemma 1).

Proof. (1) follows from Lemma 2: M-connected core points are ε-connected. (2) By (1), sending each kNN-DBSCAN cluster to the DBSCAN cluster that contains its core points defines a map between the two sets of clusters. It is onto, because every core point belongs to some kNN-DBSCAN cluster. If the two sets have the same size, it is therefore one-to-one: no DBSCAN cluster holds two kNN-DBSCAN clusters, and the clusterings of the core points coincide. □

kNN-DBSCAN can therefore *split* a cluster that DBSCAN joins through a sparse bridge, where two core points are within ε but not among each other's $M-1$ nearest neighbors, and never *merges* clusters that DBSCAN separates. In the paper's experiments, and in this project's tests on blobs, moons, anisotropic and 20-dimensional data, the clusterings agree on the core points whenever the counts agree, as part (2) guarantees.

Consequences for the graph. Only the first $M-1$ columns of each row are ever read: they decide core points (column $M-1$), cluster edges (columns $1, \dots, M-1$) and border candidates (Lemma 1, part 2). A graph stored with k neighbors therefore serves every $M \le k+1$ and every ε without recomputation, which is what makes parameter search cheap: the graph is the expensive part (Section 8).

4 Clusters as spanning forests

4.1 From components to spanning trees

Theorem 2 (Theorem 3.5). The connected components of a minimum spanning forest of $G_{M,\,\mathrm{core}}$ are the kNN-DBSCAN clusters of the core points (and those of $G_{\varepsilon,\,\mathrm{core}}$ are the DBSCAN clusters).

A spanning forest has the same components as the graph, so the statement is immediate; its value is algorithmic. Spanning forests can be built in parallel with Borůvka's algorithm, and a *minimum* spanning forest additionally supports hierarchical clustering. For clustering alone minimality is not needed, a fact the implementation relies on (Proposition 1).

4.2 Borůvka's algorithm

Borůvka's algorithm grows all trees at once. Each *subtree* (initially a single core point) selects its lightest edge to another subtree; the selected edges merge subtrees; the round repeats until no subtree has an edge left. Each round at least halves the number of subtrees that can still merge, so $O(\log n)$ rounds suffice. A round has four steps, shown in Algorithm 1 for one shared-memory process:

Minimum edges every core point proposes its lightest edge to another subtree, and each subtree keeps the lightest proposal (`pwrite`, Algorithm 3 of the paper).

Breaking symmetry each subtree u points to the target $p(u)$ of its edge, or to itself if it has none. Two subtrees that chose each other point at the one with the smaller id.

Pointer jumping $p(u) \leftarrow p(p(u))$ until nothing changes; then every subtree points to the root of its new tree.

Relabelling each core point takes the root of its subtree.

Algorithm 1: Local Borůvka over the core points of one process (paper's Algorithm 2).

Require: core points C of this process; neighbor rows sorted by distance
Ensure: $R[i]$, the root of the subtree that contains each $i \in C$

$R[i] \leftarrow i$ and $L[i] \leftarrow 1$ for every $i \in C$ *// $L[i]$: next column of row i*
repeat
  $E_{\min}[u] \leftarrow (\text{none}, +\infty)$ for every subtree u
  parallel for $i \in C$ *// minimum edges*
    while $L[i] \leq M - 1$
      $q \leftarrow$ neighbor $L[i]$ of i
      if q is stored on another rank **then** record cut edge (i, q); $L[i] \leftarrow L[i] + 1$
      else if $q \in C$ **and** $R[q] \neq R[i]$ **then** `pwrite`$(E_{\min}[R[i]], R[q], d(i, q))$; **break** *// rows are sorted*
      else $L[i] \leftarrow L[i] + 1$ *// non-core, or same subtree*
  $p(u) \leftarrow E_{\min}[u]$.target, or u if it has none *// break symmetry*
  if $p(p(u)) = u$ **and** $u < p(u)$ **then** $p(u) \leftarrow u$ *// mutual pair: smaller id is root*
  repeat $p(u) \leftarrow p(p(u))$ **until** no pointer changes *// pointer jumping*
  join the subtrees left on cycles *// Section 4.3*
  $R[i] \leftarrow p(R[i])$ for every $i \in C$ *// relabel*
until the number of subtrees does not decrease

Two details matter for cost. Rows are sorted, so the first eligible neighbor of a point is its lightest outgoing edge, and the scan stops there. And an edge that has become internal to a subtree stays internal, so the column pointer $L[i]$ only moves forward: over all rounds each of the $n(M-1)$ edges is inspected a bounded number of times.

4.3 Cycles in a directed graph

In an undirected graph with distinct weights, the selected edges contain no cycles except mutual pairs, which symmetry breaking resolves: following selections strictly decreases the weight. A kNN graph is *directed*: $q \in$

$\mathcal{N}_M(p)$ does not imply $p \in \mathcal{N}_M(q)$, so subtree u sees only the edges its own points list. The weights along a chain of selections need not decrease, and longer cycles appear (Figure 1). Ties in the weights, frequent with duplicated or quantized data, create them too.

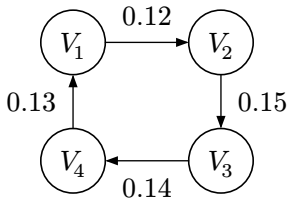


Figure 1: The directed graph of the paper's Fig. 4 ($k = 2$). Each vertex's lightest out-edge is drawn: V_2 does not list V_1, so instead of the mutual pair $V_1 \rightleftharpoons V_2$ an undirected graph would give, the selections form a 4-cycle, and pointer jumping alone cannot terminate correctly.

Pointer jumping replaces p by $p \circ p$ in every round. A vertex on a tree reaches its root after $\lceil \log_2 \text{depth} \rceil$ rounds and stays there. A vertex on a cycle of length ℓ never reaches a root, but what it does next depends on ℓ:

Cycle	1 round: p^2	2 rounds: p^4	3 rounds: p^8	Result
$a \to b \to c \to a$	$a \to c, b \to a, c \to b$	$a \to b, b \to c, c \to a$	$= p^2$	never resolved
$a \to b \to c \to d \to a$	$a \to c, b \to d, \ldots$	$\boldsymbol{a \to a, b \to b, \ldots}$	$= p^4$	four "roots"

Table 2: Pointer jumping on cycles. After t rounds each pointer is p^{2^t}, which is the identity once ℓ divides 2^t: on a cycle whose length is a power of two, every member points at itself, which is exactly how a root looks.

Power-of-two cycles. An implementation that detects cycles only by the number of unresolved subtrees no longer decreasing catches odd cycles, but on a cycle of length 2, 4, 8, … every member "resolves" to itself and becomes a root of its own. The cycle is never merged, the next round selects the same edges, no subtree merges, and Borůvka stops early with a cluster split into pieces. A subtree that was not a root and whose chain returns to itself is on a cycle and must stay unresolved, so that cycle breaking sees it. Sequential in-place updates hide the problem, because the first member to point at itself becomes the root of the others; synchronous updates (the distributed algorithm) expose it.

The subtrees left unresolved form cycles with trees hanging off them. They are joined by grouping them along their pointers into weakly connected groups and making one member of each group the root of all of it.

Grouping must use a union-find. Grouping by overwriting one representative's pointer with another's, without first following each to its root, can link representatives into a new cycle of length three or more, on which the pointer jumping that follows loops forever. A union-find with path compression cannot.

4.4 Minimum edges are not needed

The paper's minimum-edge update (`pwrite`) is a compare-and-swap loop. If the compare and the write are not one atomic operation, two threads updating the same subtree can leave an edge that is not the lightest, or even a target from one write with the weight of another. Neither affects the clusters:

Proposition 1 (Borůvka with arbitrary outgoing edges). Suppose each round, every subtree with at least one outgoing edge records *some* outgoing edge (not necessarily the lightest), and subtrees are merged along the recorded edges with cycles joined as above. Then the procedure terminates, and its final subtrees are exactly the connected components of the graph.

Proof. Safety: every merge follows an edge of the graph, so no subtree ever spans two components. *Progress:* while some component contains two subtrees, each of them has an outgoing edge and records one, and following recorded edges from any of them leads into a merge, so the number of subtrees strictly decreases.

The procedure stops only when no subtree has an outgoing edge, that is, when each subtree is a whole component. □

Minimality matters for the tree (and for hierarchical clustering), not for the clusters. The proposition also explains why the result does not depend on thread scheduling, though the spanning tree, and with it the order in which roots are found, does. Labels are therefore renumbered at the end (Section 6).

5 The distributed algorithm

5.1 Data layout

With P ranks, the points are split into contiguous blocks in rank order: with $n_0 = \lceil n/P \rceil$, rank r holds points $rn_0, \ldots, (r+1)n_0 - 1$, the last nonempty rank holds the remainder, and trailing ranks may hold nothing (7 points on 6 ranks leave ranks 4 and 5 empty). Each rank stores only its own rows, with global neighbor ids. An edge to a point of another rank is a *cut edge*.

5.2 The inexact minimum spanning tree

Theorem 3 (inexact MST, Theorem 3.6). Partition the vertices into groups. Build a minimum spanning forest of each group using only edges inside the group; contract each resulting tree to a super-vertex; build a spanning forest of the super-vertices using only the cut edges between groups. The union of these forests is generally not a minimum spanning forest of the whole graph, but its connected components are those of the graph.

Proof. Any path between two vertices alternates runs of internal edges, whose endpoints the group forests already connect, with cut edges, which the forest over super-vertices connects. Conversely every forest edge is a graph edge. □

For kNN-DBSCAN the groups are the ranks. The local stage needs no communication, and the cut stage works on far fewer vertices (subtrees) than points. The theorem also states the property an implementation must satisfy: **the clusters of core points do not depend on the number of ranks or on how points are assigned to them.**

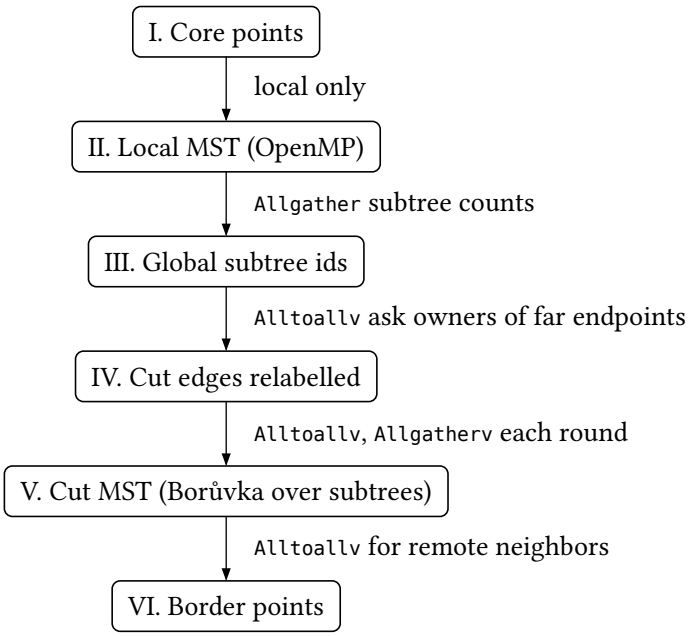


Figure 2: Stages of `knndbscan()` on each rank, with the communication each needs.

5.3 Stages

Algorithm 2 is one rank's whole run. Stages I and II need no communication; stage III gives every subtree a global id, numbering each rank's subtrees after those of lower ranks, and the rank that numbered a subtree *owns* it from then on; stage IV rewrites each cut edge between points as an edge between their subtrees,

which only the far point's rank knows. Border points may join any adjacent cluster (Section 2); one on several ranks prefers a core neighbor on its own rank, which is why border assignments, unlike clusters of core points, may differ between rank counts.

Algorithm 2: kNN-DBSCAN on one rank: `knndbscan()` (paper's Algorithm 1).

Require: this rank's rows of the kNN graph, sorted, without the point itself; ε; M
Ensure: a cluster label for each of this rank's points, -1 for noise

$C \leftarrow \{i : \text{distance in column } M-1 \text{ of row } i \le \varepsilon\}$ *// I. core points (Lemma 1)*
R, cut edges $\leftarrow$ local Borůvka over C *// II. Algorithm 1*
number this rank's subtrees after those of lower ranks *// III. `Allgather`*
rewrite each cut edge (i, j) as $(R[i], R[j])$ *// IV. `Alltoallv` to j's rank*
roots $\leftarrow$ cut MST over the subtrees *// V. Algorithm 3*
$R[i] \leftarrow$ root of $R[i]$ for every $i \in C$
parallel for each non-core point i *// VI. border points*
    $q \leftarrow$ first core neighbor of i within ε, preferring one on this rank *// `Alltoallv` if remote*
    $R[i] \leftarrow R[q]$ if q exists, otherwise -1 *// noise*

Algorithm 3: Cut MST: Borůvka over subtrees across ranks (paper's Algorithm 4).

Require: subtrees, each owned by one rank; cut edges (u, v, w) between subtree ids
Ensure: the root of every subtree

repeat
    parallel for each subtree u this rank touches: its lightest local cut edge *// local proposal*
    send each proposal to the owner of u, which keeps the lightest as $E_{\min}[u]$ *// `Alltoallv`*
    owners break symmetry between mutual pairs *// as in Algorithm 1*
    repeat *// distributed pointer jumping*
        for each unresolved u, ask the owner of $p(u)$ for $p(p(u))$ *// `Alltoallv` request and reply*
        keep u unresolved if its chain returns to u *// power-of-two cycles*
    until every subtree is resolved, **or** the unresolved count stalls
    if stalled **then** group the unresolved subtrees on rank 0 with a union-find *// `Gatherv`, `Scatterv`*
    share the new roots; relabel cut edges, dropping those inside one subtree *// `Allgatherv`*
until the global number of subtrees does not decrease

Communication and cost. The communication is collective throughout: `Allgather` and `Allgatherv` for counts and roots, `Alltoallv` for edge proposals and pointer queries, `Gatherv` and `Scatterv` for the rare cycle breaking on rank 0. The paper's analysis gives a local-stage work of $O(\frac{n}{P} \cdot M + (\frac{n}{P}) \log(\frac{n}{P}))$ per rank, and cut-stage communication that grows with the number of cut edges per rank and with $\log P$ rounds; cut edges increase with the dimension of the data, which is what limits strong scaling in high dimensions.

The library used from Python runs the same code on a single rank, through a stand-in that implements each collective for a one-rank communicator, so stages III–V degenerate to bookkeeping. The MPI command-line program runs it on any number of ranks.

6 Numerical conventions and invariants

The paper's definitions leave a few decisions to the implementation. Each of the following is fixed by the code and checked by a test.

Invariant 1 (the radius test). A distance equal to ε is within ε ($\leq$, as in Lemma 1 and in scikit-learn's DBSCAN). Distances and ε are compared in single precision, the precision of the stored graph: a double-precision ε is first rounded to the nearest `float32`.

Invariant 2 (no finite "no edge yet" weight). The weight that marks a subtree without a candidate edge is $+\infty$. Any finite sentinel s silently drops every edge of weight at least s, so the result would depend on the units of the data.

Invariant 3 (deterministic labels). Clusters are numbered $0, 1, \ldots$ in order of their lowest-indexed point, and noise is -1. The core's own numbering follows its roots, which depend on thread scheduling (Proposition 1); renumbering makes the labels a function of the partition alone, identical for any number of threads.

Invariant 4 (invariances). The clustering is unchanged by scaling all distances and ε by the same factor, by any strictly increasing transformation of distances and ε (the algorithm only compares them), by permuting the points (which permutes the labels), and by dropping graph columns beyond $M - 1$. The clusters of core points are unchanged by the number of threads and ranks (Theorem 3).

Invariant 5 (valid input). Each row lists neighbors other than the point, in nondecreasing order of finite, nonnegative distance, with $2 \leq M \leq k + 1$. The algorithm relies on all of it: border labelling stops at the first distance beyond ε and depends on sorted rows to stay inside its row, and an out-of-range index is read as another point's row. Before clustering, the library rejects any row with an index out of range, the point itself, or a distance that is negative, not finite or out of order, as well as an M outside that range.

7 Verifying an implementation

Clustering output has no simple "expected value", so the project checks the implementation against the paper's statements rather than against fixed outputs.

Reference implementation the definitions of Section 3 computed directly with sparse connected components (`tests/reference.py`), written from the paper, never from the C++ code. The library must reproduce its core points, clusters and noise exactly, and give each border point an allowed cluster.

Random directed graphs graphs whose distances obey no triangle inequality and whose edges are rarely mutual, with ties and zero distances. They make Borůvka meet the cycles of Section 4.3 far more often than real data does, and they were the most effective tests: they found every defect in the cycle handling.

Theorems as tests core, border and noise points equal scikit-learn's DBSCAN (Lemma 1), clusters refine DBSCAN's and coincide when the counts match (Theorem 1), and growing ε only adds core points and merges clusters.

Metamorphic tests the invariances of Section 6, applied exactly: scaling by powers of two (exact in floating point, up to $2^{\pm 120}$), integer distances under affine and quadratic maps, random permutations, truncated graphs.

Ranks and threads the MPI program on 1 to 7 ranks, with points shuffled (most edges cut) or grouped by cluster (few cut), with ranks that own no points, compared with the reference (Theorem 3); repeated runs and thread counts must agree.

Sanitizers the library, the MPI program and C tests built with AddressSanitizer and UndefinedBehaviorSanitizer and run against all of the above. Sanitized builds change memory layout and timing, which exposed a data race that optimized builds hid.

The original implementation, which produced the paper's results, passed its own experiments; the tests above found the following defects, all fixed in this implementation. None is in the method; each is a way the method's assumptions can be broken in code.

Defect	Effect	Found by
Strict comparison $d < \varepsilon$	points exactly at ε were not core (grid with $\varepsilon = 1$: none instead of 16)	Lemma 3.1 test on a grid
Finite sentinel weight 10^4	edges of weight $\geq 10^4$ never merged (data scaled by 2^{16}: 109 clusters instead of 3)	scaling by powers of two
Power-of-two cycles in distributed pointer jumping	clusters split on some rank counts (one cluster became 6 on 7 ranks)	random graphs over rank counts
Grouping by overwriting representatives	endless loop in cycle breaking (5 ranks)	random graphs over rank counts
Local cycle grouping read past its array	subtrees on cycles merged into point 0's cluster	AddressSanitizer
Loop variable shared by threads	core points labelled with another thread's root	sanitized builds, thread sweep
Every rank truncated the output file	empty labels file on oversubscribed ranks	MPI tests on CI runners

Table 3: Defects found in the original implementation by the tests of Section 7.

A lesson for testing parallel graph code. All the cycle defects lived in paths that real data rarely reaches: exact kNN graphs of continuous data produce mostly mutual pairs, and a few ranks rarely produce cycles whose length is a power of two. Randomized inputs that are valid but adversarial, swept over thread and rank counts and compared with an independent reference, reached them within seconds.

8 Performance

The paper reports, on TACC's Frontera, one billion 3-D points clustered in under a second on 28K cores, 65 billion 20-D points in under 40 s on 114,688 cores, and runs up to 37× faster than PDBSCAN [5] on 4 million 20-D points. Strong scaling of 64 million points from 4 to 256 ranks gives speedups of 13.8 (3-D), 16.8 (5-D) and 22.2 (32-D). Two observations shape how the method is used:

- **The graph dominates.** Building the k-NNG of 64 million 5-D points with $k = 100$ takes about 400 times longer than clustering it, so the graph should be computed once and reused across ε and M (Section 3). Approximate graphs are acceptable: kNN-DBSCAN is defined on whatever graph it is given, and in the paper's experiments lowering the graph's accuracy to 75 % did not change the clustering quality.
- **Cut edges limit scaling.** With more ranks, local work shrinks but the fraction of cut edges, and the communication per Borůvka round, grows; it grows with dimension too.

On a single node, the clustering stage of the Python package takes about 0.9 s for a 4-million-point graph with 19 neighbors per point on one thread and 0.2–0.26 s on eight, with no further gain at 16 on a 16-thread laptop. Checking the input graph and renumbering labels add 5–10 %. Logging costs nothing when disabled.

9 Using it

Choose M first, as for DBSCAN (larger M smooths the density estimate); then choose ε from the sorted distances to each point's $(M-1)$-th neighbor, at the elbow of that curve [6]. Compute a graph with $k \geq M - 1$, larger if several M will be tried, and cluster it for each candidate:

```
from knndbscan import KNNDBSCAN, knn_dbscan, knn_dbscan_from_graph

labels = knn_dbscan(X, eps=0.05, min_samples=15, n_jobs=-1)           # exact graph
labels = knn_dbscan_from_graph(neighbors, distances, eps=0.05, min_samples=15)
model = KNNDBSCAN(eps=0.05, min_samples=15).fit(X)   # labels_, core_sample_indices_
```

For graphs that do not fit on one node, the MPI program reads a graph file and runs on any number of ranks:

```
mpiexec -n 64 knndbscan -n N -e eps -m M -k K -t threads -i graph.txt -o labels.txt
```

References

[1] Y. Chen, W. Ruys, and G. Biros, "KNN-DBSCAN: a DBSCAN in high dimensions," 2024. [Online]. Available: https://arxiv.org/abs/2009.04552

[2] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," in *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD)*, 1996, pp. 226–231.

[3] B. Xiao and G. Biros, "Parallel algorithms for nearest neighbor search problems in high dimensions," *SIAM Journal on Scientific Computing*, vol. 38, no. 5, 2016.

[4] C. D. Yu, S. Reiz, and G. Biros, "Distributed-memory hierarchical compression of dense SPD matrices," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis (SC18)*, 2018.

[5] M. M. A. Patwary, D. Palsetia, A. Agrawal, W.-keng Liao, F. Manne, and A. Choudhary, "A new scalable parallel DBSCAN algorithm using the disjoint-set data structure," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis (SC12)*, 2012.

[6] E. Schubert, J. Sander, M. Ester, H.-P. Kriegel, and X. Xu, "DBSCAN revisited, revisited: why and how you should (still) use DBSCAN," *ACM Transactions on Database Systems*, vol. 42, no. 3, 2017.

A The algorithm in pictures

This appendix shows the algorithm at work on a small two-dimensional example, stage by stage, with each panel tied to the lines of 1, 2 and 3 it illustrates, and then shows, step by step, how each defect of Table 3 arises. The data figures are computed, not drawn: `docs/report/figures/make_figures.py` replays Borůvka's rounds exactly as Algorithm 1 states them and checks that the replay finds the library's clusters before drawing; final clusterings come from the library and from scikit-learn's DBSCAN. The example has three blobs, $M = 6$ and $\varepsilon = 0.55$.

A.1 From points to clusters

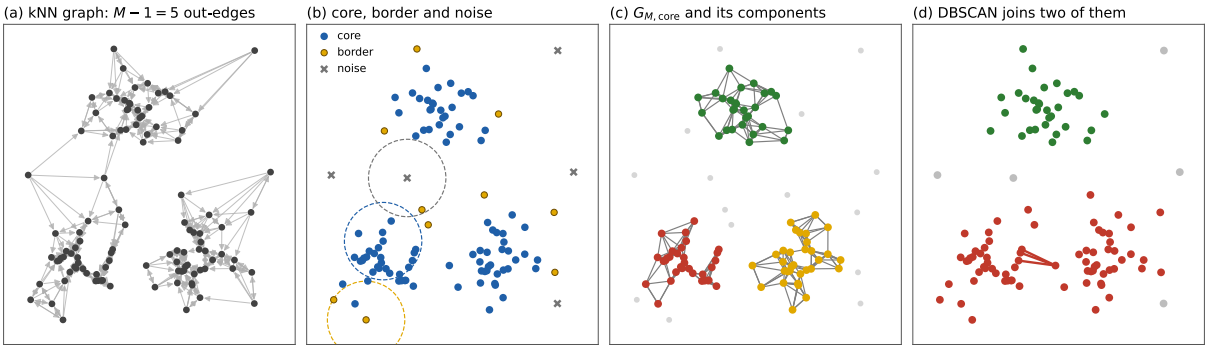


Figure 3: The definitions on the example. (a) Each point's $M-1$ nearest neighbors (arrows); the graph is directed. (b) The classes of Lemma 1: a point is core when its ε-disk (dashed) holds M points; the border point lies within ε of a core point, the noise point of none. (c) The undirected graph $G_{M,\,\mathrm{core}}$ and its components, which are the clusters of kNN-DBSCAN. (d) DBSCAN on the same data: the red pairs are core points within ε of each other but not among each other's $M-1$ nearest neighbors, so DBSCAN joins two blobs that kNN-DBSCAN keeps apart, the split that Theorem 1 allows.

Panel (b) is line 1 of Algorithm 2; the components in (c) are what lines 2–6 compute; the border and noise points are labelled by lines 7–9.

A.2 Borůvka, round by round

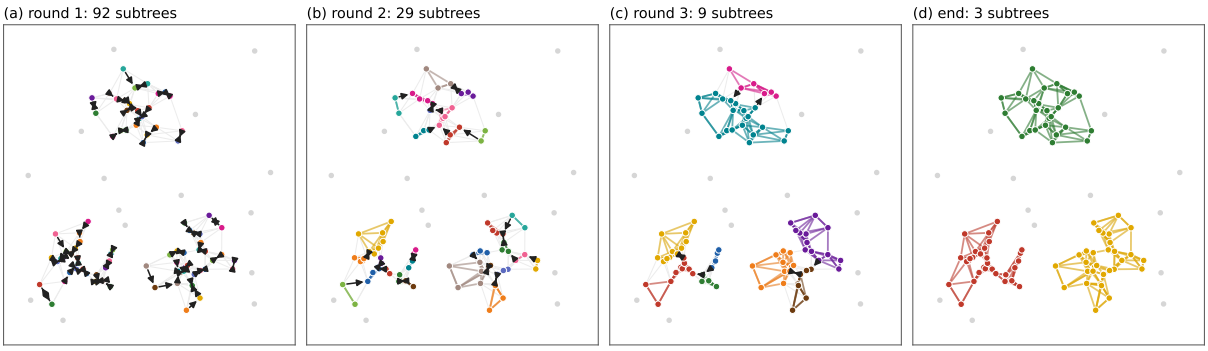


Figure 4: Algorithm 1 on the core points of the example. Arrows are the edges selected in a round (lines 4–9: each subtree's lightest edge to another subtree); colors are the subtrees at the start of the round, and faint lines the edges of $G_{M,\,\mathrm{core}}$. Double-headed arrows are mutual pairs, resolved by line 11. Pointer jumping and relabelling (lines 12–14) turn the selections into the next round's subtrees: 92, 29, 9 and finally 3, the clusters, after which no subtree has an edge left (line 15).

Each round at least halves the number of subtrees that can still merge, so the number of rounds grows like $\log n$; on this example the counts fall $92 \to 29 \to 9 \to 3$.

A.3 Across ranks

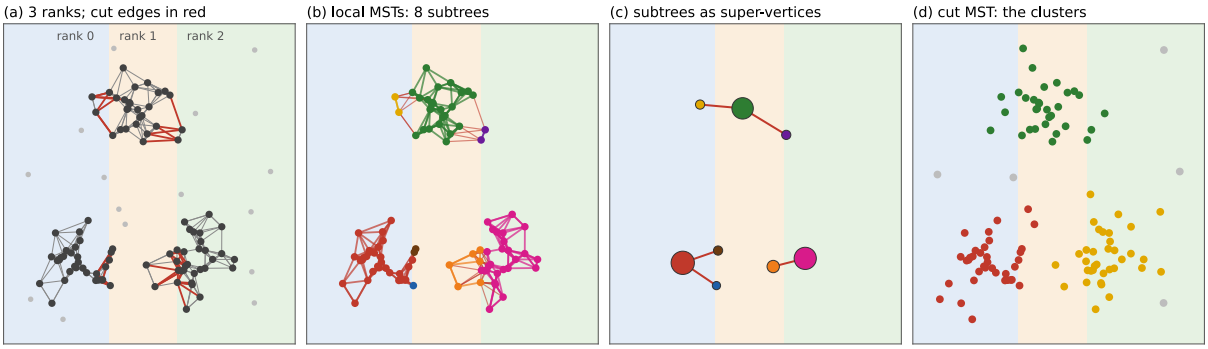


Figure 5: The example on three ranks, with points numbered left to right so that each rank's block is a vertical strip. (a) Edges of $G_{M,\,\mathrm{core}}$ inside a rank (grey) and cut edges between ranks (red). (b) Stage II, local Borůvka without the cut edges: 8 subtrees, some of them single points. (c) Stages III–IV: each subtree becomes one super-vertex, joined by the cut edges between them. (d) Stage V, the cut MST over the super-vertices: its components are the same three clusters, as Theorem 3 guarantees for any split.

Panel (b) is line 2 of Algorithm 2, (c) lines 3–4, and (d) Algorithm 3. The more cut edges a split creates, the more of the work moves from the communication-free stage II to the communicating stage V, which is the scaling limit noted in Section 8.

A.4 Where implementations go wrong

Each item below is a row of Table 3, drawn as it happens.

A.4.1 Cycles that look like roots

Pointer jumping replaces every pointer p by $p \circ p$ in each round (line 12 of Algorithm 1, lines 5–8 of Algorithm 3). On the four-cycle of Figure 1, every pointer reaches its own node after two rounds:

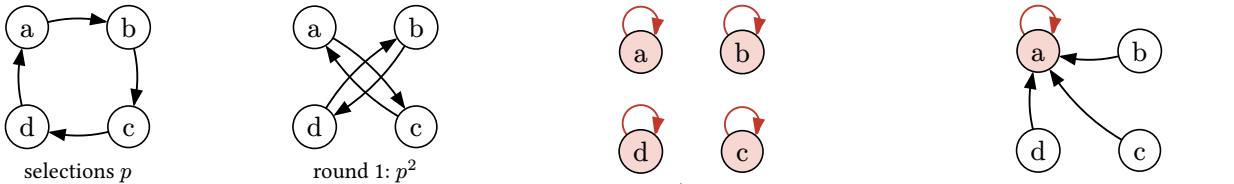


Figure 6: Pointer jumping on a 4-cycle. Every member ends pointing at itself, which is how a root looks (shaded). Taken as roots, the four subtrees never merge, the next round selects the same edges, and Borůvka stops with the cluster in four pieces. With line 7 of Algorithm 3, a member whose chain returns to itself stays unresolved, and cycle grouping makes one member the root of all (right).

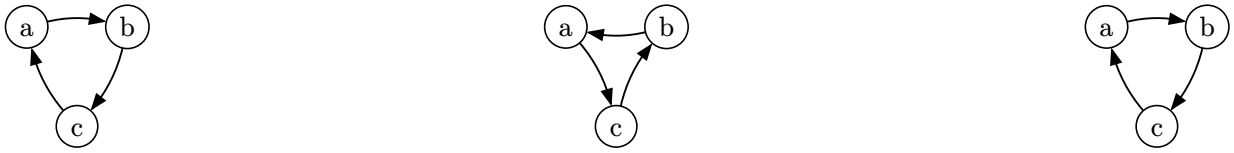


Figure 7: On a 3-cycle the pointers rotate forever and never reach a node's own id, so the number of unresolved subtrees stalls: this is the case a stall-based check catches (lines 8–9 of Algorithm 3). Only cycles whose length is a power of two escape it.

The original implementation found the clusters on every rank count except some where the selections happened to form a power-of-two cycle; the test that swept random directed graphs over 1 to 7 ranks found one such case on 7 ranks.

A.4.2 Grouping that creates a cycle

The subtrees left unresolved are grouped along their pointers (line 13 of Algorithm 1, line 9 of Algorithm 3). Merging two groups by overwriting one representative's pointer with the other's current representative, without first following each pointer to its root, can close a loop among representatives:



overwriting pointers: $r_1 \to r_2 \to r_3 \to r_1$ union-find: find both roots, then link root to root

Figure 8: Grouping. Linking a group to another without finding its root first lets a chain of representatives become a cycle, and the pointer jumping that follows grouping loops forever: the run on 5 ranks that never finished. A union-find links only roots, so the representatives form a tree with a single root (shaded).

A third defect sat in the same place in the shared-memory code: the local grouping read one element past its list of unresolved subtrees, formed no groups, and pointed every one of them at the subtree of point 0, merging them into whatever cluster point 0 was in. AddressSanitizer reported the read; only multithreaded runs reach this code, because sequential in-place pointer updates collapse cycles on their own.

A.4.3 Distances exactly at ε

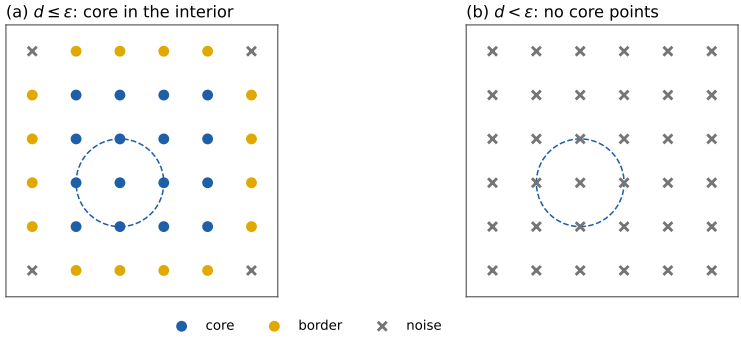


Figure 9: A unit grid with $\varepsilon = 1$ and $M = 5$: each interior point has four neighbors at distance exactly ε (dashed circle). (a) With $d \le \varepsilon$, as in Lemma 1 and in scikit-learn, interior points are core and the rest border or noise. (b) With the strict test $d < \varepsilon$ of the original implementation, no point is core. Data on grids, and integer or rounded distances, put many points exactly at ε.

A.4.4 A finite "no edge" weight

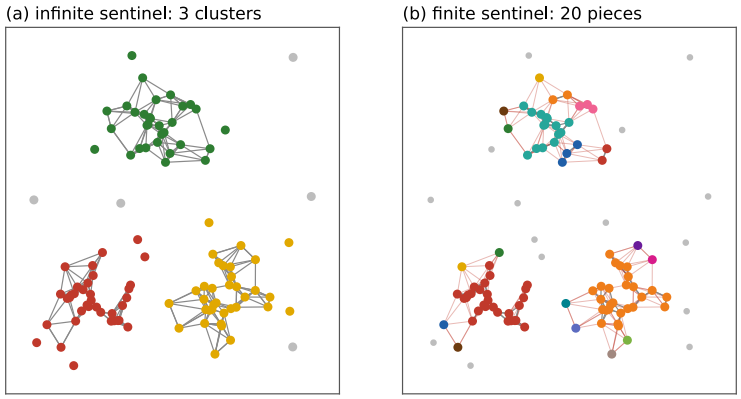


Figure 10: The minimum-edge search starts every subtree at a "no edge yet" weight s and keeps only lighter edges (line 3 of Algorithm 1). With $s = +\infty$ every edge can be selected (a). With a finite s, edges of weight s or more (faint red) are never selected, and clusters fall apart into the pieces held together by lighter edges (b). The original $s = 10^4$ did this to any data whose neighbor distances reach 10^4, for instance in raw physical units; here s is set to the 60th percentile of the example's edge weights to show the same effect at this scale.

A.4.5 Timing: a shared variable and a shared file

Two defects depended only on how threads or ranks were scheduled, which is why they appeared under sanitizers (slower, differently timed builds) and on oversubscribed CI machines but not in ordinary runs.

	thread A	thread B	shared u
1	$u \leftarrow R[p]$		$R[p]$
2		$u \leftarrow R[q]$	$R[q]$
3	$R[p] \leftarrow \mathrm{root}(u)$		$R[q]$

	rank 0	rank 2 (late)
1	open, truncate	*still computing*
2	write n labels	
3		open, truncate

Figure 11: Left: the loop that labels core points with their cluster (line 6 of Algorithm 2) used one variable u for all threads, so point p could receive the cluster of thread B's point q; declaring u inside the loop fixes it. Right: every rank of the command-line program opened, and so emptied, the output file, but only rank 0 wrote it; a rank that reached the open after rank 0 finished left an empty file. Only rank 0 may open it.